ZENIKA · FORMATION IA · 1H30

Sous le capot

LLM, SLM, agents, harnais, orchestrateurs : comment ça marche vraiment

Du prompt tapé dans un agent de code jusqu'au fichier modifié sur le disque.

À la fin de la session, vous saurez…

LE FIL ROUGE

Suivre une requête de bout en bout

De votre prompt jusqu'au fichier modifié : quelles briques interviennent, dans quel ordre, et où se jouent **la qualité, le coût et la sécurité**.

prompt · gateway · harnais · LLM · outil · résultat

Nommer

Distinguer agent, LLM, SLM, harnais, orchestrateur et SDD.

Expliquer

Tokens, embeddings, couches, attention, KV cache.

Décortiquer

La boucle d'un agent et ses appels d'outils.

Choisir

Un harnais, un orchestrateur, un modèle adapté.

Sécuriser

Guardrails et gateways : qui fournit quoi.

© ZENIKA 2026 | All rights reserved

Agenda

1. Ouverture — 3 min
2. Cartographie — 15 min
3. LLM & SLM sous le capot — 30 min
4. Anatomie d’un agent — 15 min
5. Harnais & orchestrateurs — 12 min
6. Guardrails & gateways — 15 min
7. Synthèse — 2 min

© ZENIKA 2026 | All rights reserved

Partie 1 · 15 min

Cartographie

Six mots qu’on mélange : une définition et des produits du marché pour chacun.



© ZENIKA 2026 | All rights reserved

Six notions, une phrase chacune

LLM · LARGE LANGUAGE MODEL

Le moteur

Un réseau de neurones de dizaines à centaines de milliards de paramètres qui prédit le token suivant.

GPT-5 · Claude · Gemini · Mistral Large · DeepSeek

SLM · SMALL LANGUAGE MODEL

Le petit moteur

Même architecture, 1 à ~15 milliards de paramètres. Tourne sur un laptop, un téléphone, un petit GPU.

Phi-4 · Gemma 3 · Llama 3.2 3B · Qwen3 · Ministral

AGENT

Le modèle qui agit

Un modèle placé dans une boucle avec des outils et un objectif : il décide lui-même de la prochaine action.

Copilot coding agent · Devin · Codex

HARNAIS · HARNESS

Tout sauf le modèle

Le code qui entoure le modèle : boucle, outils, contexte, permissions, sandbox, hooks.

Claude Code · Codex CLI · Gemini CLI · OpenCode

ORCHESTRATEUR

Le chef d'orchestre

Coordonne plusieurs agents ou étapes : routage, parallélisme, état partagé, reprise sur erreur.

LangGraph · CrewAI · MS Agent Framework · Google ADK

SDD · SPEC-DRIVEN DEVELOPMENT

La méthode

Une démarche, pas un logiciel : la spec écrite devient la source de vérité que les agents exécutent.

GitHub Spec Kit · Kiro · OpenSpec · BMAD

© ZENIKA 2026 | All rights reserved

Une analogie qui tient la route

L'IMAGE À GARDER

Une flotte de véhicules de livraison

Le **moteur** ne livre rien tout seul. Il faut un châssis, un tableau de bord, des freins, un chauffeur, un dispatch et un bon de commande.

On filera la métaphore tout au long de la session.

LLM / SLM

Le moteur

Puissant ou économe. Ne sait que tourner.

HARNAIS

Le véhicule

Châssis, volant, freins, rétroviseurs.

AGENT

Le véhicule en tournée

Moteur + véhicule + une mission.

ORCHESTRATEUR

Le dispatch

Répartit les tournées entre véhicules.

SDD

Le bon de commande

Ce qu'on livre, à qui, et comment vérifier que c'est livré.

BONUS PARTIE 5

Guardrails = glissières · Gateway = péage

On y passe tous, on y contrôle et on y compte.

© ZENIKA 2026 | All rights reserved

Qui parle à qui : la pile complète

Méthode : SDD spec.md → plan.md → tasks.md, relus par l'humain

Orchestrateur découpe, distribue, attend, agrège, reprend sur erreur

Agent A	Agent B	Agent C
harness : boucle, tools, contexte	harness : boucle, tools, contexte	harness : boucle, tools, contexte
appel au modèle	appel au modèle	appel au modèle

Gateway + guardrails auth, routage, quotas, cache, journalisation, filtrage entrée / sortie

LLM cloud	Hyperscaler	Souverain	SLM local
Anthropic, OpenAI...	*Bedrock, Vertex, Azure*	*Mistral, Scaleway...*	*Ollama, vLLM*

L'humain écrit et valide la spec, puis supervise aux points clés.

Outils

fichiers, terminal, git, navigateur

serveurs MCP : GitHub, Jira, BDD...

exécutés par le harness, jamais par le modèle

*Point de passage obligé : c'est là que l'entreprise **contrôle**.*

© ZENIKA 2026 | All rights reserved

Chatbot, workflow ou agent ?

	Chatbot	Workflow LLM	Agent
Qui décide de l'étape suivante ?	l'humain	le code (chemin écrit à l'avance)	le modèle
Outils	aucun ou peu	appelés à des étapes fixes	choisis à chaque tour
Appels au modèle	1 par message	nombre fixe	variable, inconnu à l'avance
Prévisibilité / coût	élevée / faible	élevée / faible	faible / variable
Exemple	ChatGPT, Le Chat	résumer chaque ticket Jira entrant	« Corrige ce bug et ouvre la PR »

i Règle d'Anthropic (*Building effective agents*) : commencer par **la solution la plus simple** qui marche. Un workflow prévisible bat souvent un agent autonome.

© ZENIKA 2026 | All rights reserved

Harnais vs orchestrateur : la frontière

HARNAIS

Fait d'un modèle un agent

- Gère une boucle de raisonnement
- Exécute les outils, applique les permissions
- Construit et compacte le contexte
- Interface : terminal, IDE, API

Claude Code | Codex CLI | Copilot CLI | Aider | OpenHands

ORCHESTRATEUR

Fait travailler plusieurs agents ensemble

- Décompose et route les tâches
- Parallélise, attend, agrège
- Persiste l'état, reprend après panne
- Interface : code, graphe, YAML

LangGraph | CrewAI | OpenAI Agents SDK | Temporal

La frontière bouge : les harnais lancent des sous-agents (Claude Code, Copilot, Codex), et les orchestrateurs embarquent leur propre boucle d'agent. Retenez le **rôle**, pas le produit.

© ZENIKA 2026 | All rights reserved

Quiz : qui est qui ?

Ollama runtime local	Gemma 3 4B SLM	Claude Sonnet LLM	Claude Code harnais
LangGraph orchestrateur	GitHub Spec Kit outillage SDD	LiteLLM gateway	Llama Guard guardrail
MCP protocole	Copilot coding agent agent cloud	text-embedding-3 embedding	vLLM serveur d'inférence

© ZENIKA 2026 | All rights reserved

Partie 2 · 30 min

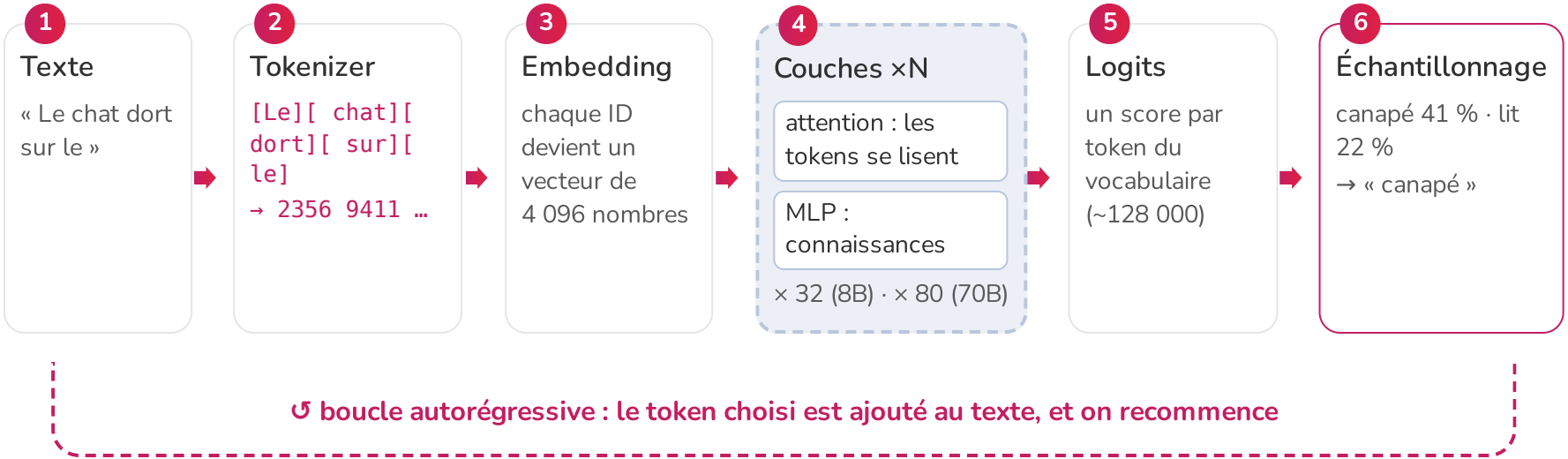
LLM & SLM sous le capot

On suit la phrase « Le chat dort sur le » jusqu'au mot suivant.



© ZENIKA 2026 | All rights reserved

Le trajet d'une phrase dans le modèle



Ordres de grandeur pour **Llama 3 8B** : vocabulaire 128 256 tokens · vecteurs de 4 096 dimensions · 32 couches · 8 milliards de paramètres

i Un LLM ne génère **qu'un seul token à la fois** : pour 500 tokens de réponse, il refait ce trajet 500 fois.

© ZENIKA 2026 | All rights reserved

Tokenisation : le modèle ne lit pas des mots

Mot rare `Anti` `constitution` `nellement` — 1 mot → **3 tokens**

Code `def` ` get` `_user` `(id` `):` — 4 mots → **5 tokens**

ALGORITHME

BPE

Byte Pair Encoding : on fusionne les paires de caractères les plus fréquentes du corpus jusqu'à obtenir le vocabulaire voulu.

~4

caractères par token en anglais. Le français et le code consomment souvent plus de tokens pour le même sens.

128 k

tokens dans le vocabulaire de Llama 3 ; ~200 k pour les tokenizers récents d'OpenAI.

i On **facture** et on **limite** en tokens, pas en mots. « Combien de r dans strawberry ? » : le modèle ne voit pas les lettres.

© ZENIKA 2026 | All rights reserved

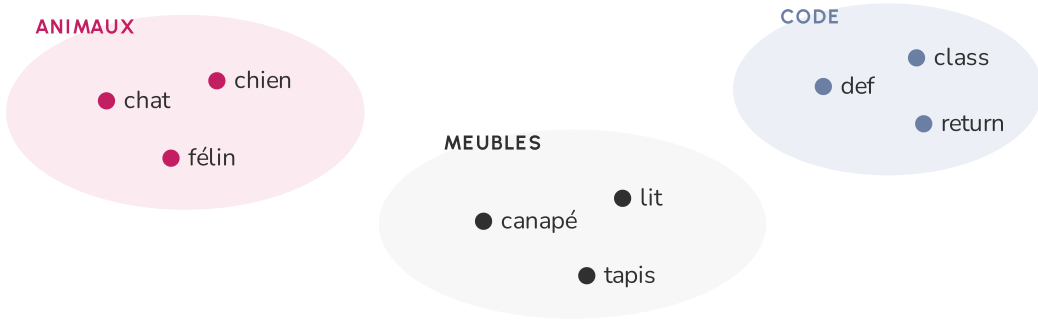
Embeddings : du numéro au sens

L'IDÉE

Chaque token devient un point dans un espace à des milliers de dimensions

La **table d'embedding** est apprise : des mots au sens proche finissent à des positions proches.

> *Llama 3 8B : 128 256 tokens × 4 096 dimensions ≈ 525 millions de paramètres, rien que pour cette table.*



projection 2D d'un espace à 4 096 dimensions (illustratif)

Deux sens du mot : l'**embedding interne** est la première couche d'un LLM, invisible pour vous. Un **modèle d'embedding** transforme un texte entier en vecteur pour la recherche sémantique et le RAG (`text-embedding-3` , Mistral Embed, Voyage, Cohere, BGE, nomic-embed).

© ZENIKA 2026 | All rights reserved

Une couche transformer, c’est deux opérations

① ATTENTION

Les tokens se lisent entre eux

Chaque token va chercher l'information utile dans les tokens précédents.

Dans « le chat ne traverse pas car il est fatigué », *il* regarde fortement *chat*.

> *Plusieurs « têtes » en parallèle : l'une suit la syntaxe, l'autre les références…*

② MLP / FEED-FORWARD

Chaque token réfléchit seul

Un réseau dense appliqué à chaque position. C'est là que se loge l'essentiel des connaissances factuelles.

> *Environ deux tiers des paramètres d'un modèle dense.*

LES LIANTS

Connexion résiduelle : chaque couche *ajoute* au vecteur au lieu de le remplacer. Normalisation : garde les valeurs stables.

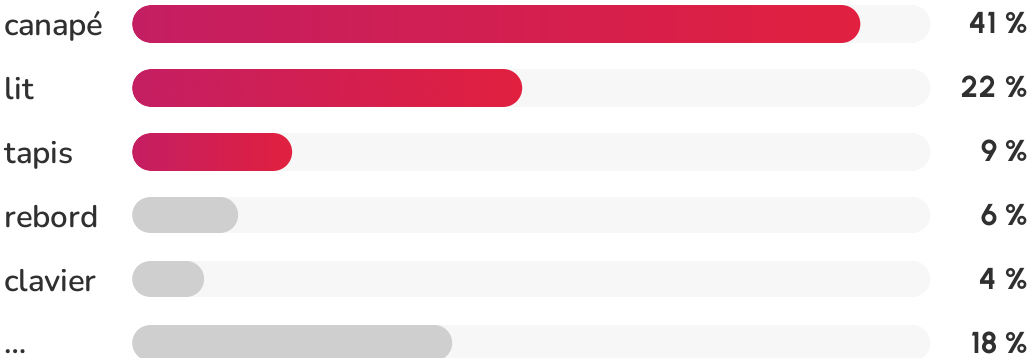
EMPILEMENT

On répète le bloc 32 fois (8B), 80 fois (70B), 126 fois (405B). Les premières couches captent la forme, les suivantes le sens.

© ZENIKA 2026 | All rights reserved

Choisir le mot suivant : logits, température, top-p

Après « Le chat dort sur le »



Les logits (scores bruts) passent par un softmax qui les transforme en probabilités. En couleur : le noyau retenu avec top_p = 0.7.

TEMPÉRATURE

Basse (0 à 0,3) : on prend presque toujours le favori. Haute (> 1) : on aplatit, plus de surprises.

TOP-P / TOP-K

On ne tire qu'au sein des candidats les plus probables (cumul ≥ p, ou les k premiers).

i Un LLM produit ce qui est **plausible**, pas ce qui est **vrai**. L'hallucination n'est pas un bug ponctuel : c'est le fonctionnement normal quand le contexte ne contient pas la bonne information.

© ZENIKA 2026 | All rights reserved

D’où viennent les poids : trois phases d’entraînement

1

Pré-entraînement : lire Internet

Prédire le token suivant sur des milliers de milliards de tokens (15 000 milliards pour Llama 3). Des semaines sur des milliers de GPU.

RÉSULTAT

un modèle « de base » qui complète du texte

2

Fine-tuning supervisé : apprendre à répondre

Des exemples question / bonne réponse, des conversations, des appels d’outils.

RÉSULTAT

un modèle « instruct » qui suit des consignes

3

Renforcement : apprendre à bien répondre

Préférences humaines (RLHF), ou récompenses vérifiables : le test passe, le calcul est juste.

RÉSULTAT

les modèles « de raisonnement » et agentiques

i **Date de coupure** : la librairie sortie le mois dernier, il faut la **fournir dans le contexte**. **Tokens de raisonnement** : le modèle « réfléchit » en générant des tokens, facturés et stockés dans le contexte.

© ZENIKA 2026 | All rights reserved

La fenêtre de contexte : la seule mémoire de travail

À RETENIR

Le modèle n'a pas de mémoire entre deux appels

À chaque tour, le harnais **renvoie tout** : instructions, historique, fichiers lus, résultats d'outils. Le modèle relit tout depuis le début.

> *« Mémoire » d'un agent = ce que le harnais choisit de remettre dans la fenêtre.*

128 k

tokens : la norme des modèles ouverts récents

1 M+

tokens : offres longues de Gemini, Claude, GPT



PLUS LONG ≠ MIEUX

Les performances se dégradent quand le contexte s'allonge (*lost in the middle*, *context rot*) : l'information au milieu est moins bien exploitée.

ORDRE DE GRANDEUR

128 k tokens ≈ un roman de 300 pages, ou ~10 000 lignes de code. Un agent qui lit 30 fichiers et relance 10 fois les tests atteint vite cette limite : d'où la compaction (résumé automatique) dans les harnais.

© ZENIKA 2026 | All rights reserved

KV cache : ne pas tout recalculer à chaque token

1 Prefill : lire le prompt

Tous les tokens du prompt traités en parallèle.

`Le` `chat` `dort` `sur` `le`

KV cache : K et V de chaque token

- limité par le calcul (GPU saturé)
- fixe le temps avant le premier token (TTFT)

2 Decode : écrire la réponse

Un seul nouveau token par passe.

`Le` `chat` `dort` `sur` `le` **`canapé`**

cache réutilisé tel quel + K, V de « canapé »

- limité par la bande passante mémoire
- fixe les tokens par seconde

SANS CACHE

Le token n° 1 000 recalculerait K et V des 999 précédents. Le cache transforme un coût quadratique en coût linéaire.

LE PRIX

Llama 3 8B en 16 bits : 128 Ko par token. Pour 128 k tokens : 16 Go, autant que le modèle lui-même.

LES PARADES

GQA (têtes K/V partagées), cache quantifié, PagedAttention (vLLM) pour paginer la mémoire comme un OS.

© ZENIKA 2026 | All rights reserved

Du KV cache au prompt caching : la facture de vos agents

LE PRINCIPE

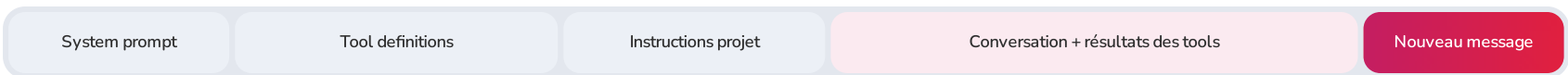
Les fournisseurs gardent le KV cache d'un préfixe déjà vu

Si votre requête commence **exactement** comme la précédente, le prefill de cette partie est sauté : réponse plus rapide, tokens d'entrée facturés beaucoup moins cher.

~90 %

de réduction possible sur les tokens d'entrée lus en cache (ordre de grandeur)

ANATOMIE D'UNE REQUÊTE D'AGENT : DU PLUS STABLE AU PLUS VOLATIL



← *stable : servi par le cache d'un tour à l'autre*

volatil : seule la fin est recalculée →

ANTHROPIC

Points de cache explicites (`cache_control`), lecture facturée ~10 % du prix d'entrée.

OPENAI

Cache automatique des préfixes longs, remise appliquée sans configuration.

GOOGLE GEMINI

Cache implicite + *context caching* explicite avec durée de vie.

© ZENIKA 2026 | All rights reserved

Rendre un modèle plus léger : quatre techniques



Quantification

Stocker les poids sur moins de bits : 16 → 8 → 4 bits. Perte de qualité faible jusqu'à 4 bits, nette en dessous.

> *GGUF (llama.cpp, Ollama), AWQ, GPTQ, MLX*



Distillation

Un grand modèle « professeur » génère des réponses ; un petit modèle « élève » apprend à les imiter.

> *La plupart des SLM récents sont distillés.*



Élagage

Retirer des couches ou des neurones peu utiles, puis ré-entraîner un peu.



Mixture of Experts

Pas plus petit, mais moins actif : seule une partie des poids travaille pour chaque token.

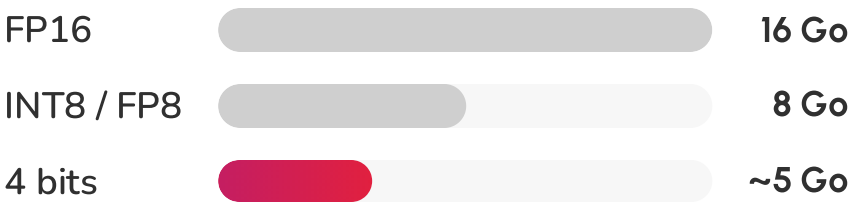
> *Un routeur choisit quelques experts par token.*

i Quantification et distillation sont les deux ingrédients principaux des SLM.

© ZENIKA 2026 | All rights reserved

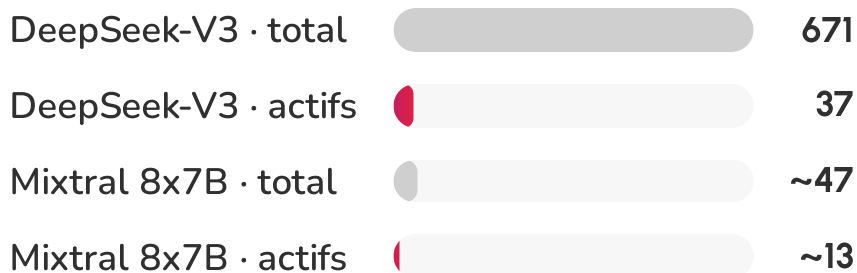
Ce que pèse un modèle

Quantification : mémoire d'un modèle 8B



Mémoire des poids ≈ paramètres × octets par paramètre, plus le KV cache.

MoE : paramètres totaux vs actifs (milliards)



Le calcul suit les paramètres actifs, mais tous les poids doivent tenir en mémoire.

Un modèle 8B quantifié en 4 bits tient sur un MacBook avec **16 Go** de mémoire unifiée.

© ZENIKA 2026 | All rights reserved

Les SLM : petits, locaux, spécialisés

DÉFINITION PRATIQUE

De ~1 à ~15 milliards de paramètres

Assez petit pour tourner sur un poste, un téléphone ou un seul GPU. Même architecture qu'un LLM, juste moins de couches et des vecteurs plus étroits.

LE MARCHÉ

Phi-4 (14B), Phi-4-mini (3,8B) · Gemma 3 (1B à 27B) · Llama 3.2 (1B, 3B) · Qwen3 (0,6B à 14B) · Ministral (3B, 8B) · SmolLM

POUR FAIRE TOURNER

Poste : Ollama, LM Studio, llama.cpp, MLX. Serveur : vLLM, SGLang, TGI.

EMBARQUÉ

Gemini Nano (Android, Chrome), Apple Foundation Models (~3B, sur l'appareil).

BONS CAS D'USAGE

Classer, extraire, router, résumer court, compléter du code, données sensibles qui ne doivent pas sortir, latence faible, coût marginal nul.

LIMITES

Raisonnement long multi-étapes, connaissances générales larges, agents autonomes sur de grosses bases de code : le LLM frontière reste devant.

PATTERN HYBRIDE

Le SLM trie et route, le LLM raisonne. Un guardrail est souvent lui-même un SLM (Llama Guard, Prompt Guard).

© ZENIKA 2026 | All rights reserved

Ce que la mécanique change pour un développeur

TOKENS

Comptez en tokens

Coût, limites et latence s'expriment en tokens. Le français et le code verbeux coûtent plus.

AUTORÉGRESSIF

Un token à la fois

La longueur de la sortie fait la latence. Demandez des réponses courtes et des diffs.

SANS ÉTAT

Tout est dans le contexte

Ce qui n'est pas dans la fenêtre n'existe pas. D'où la doc, `AGENTS.md`, le RAG.

KV CACHE

Gardez le préfixe stable

Ne touchez pas aux instructions et aux outils en cours de session : le cache vous fait économiser.

PLAUSIBLE

Vérifiez

Le modèle prédit le probable. Tests, typage et revue restent les garde-fous de la vérité.

TAILLE

Le bon modèle au bon endroit

SLM local pour trier et protéger, LLM frontière pour raisonner.

© ZENIKA 2026 | All rights reserved

Partie 3 · 15 min

Anatomie d’un agent

Un modèle qui n’écrit que des tokens finit par lire des fichiers, lancer des tests et ouvrir une PR.



© ZENIKA 2026 | All rights reserved

Agent = modèle + outils + boucle + objectif

LA PHRASE CLÉ

Le modèle n'exécute rien. Il écrit une demande d'action.

Le harnais lit cette demande, l'exécute (ou la refuse), et renvoie le résultat au modèle. On recommence jusqu'à ce que le modèle réponde sans demander d'outil.



Modèle

Décide de la prochaine action.



Outils

Lire, écrire, exécuter, chercher.



Boucle

Agir, observer, recommencer.



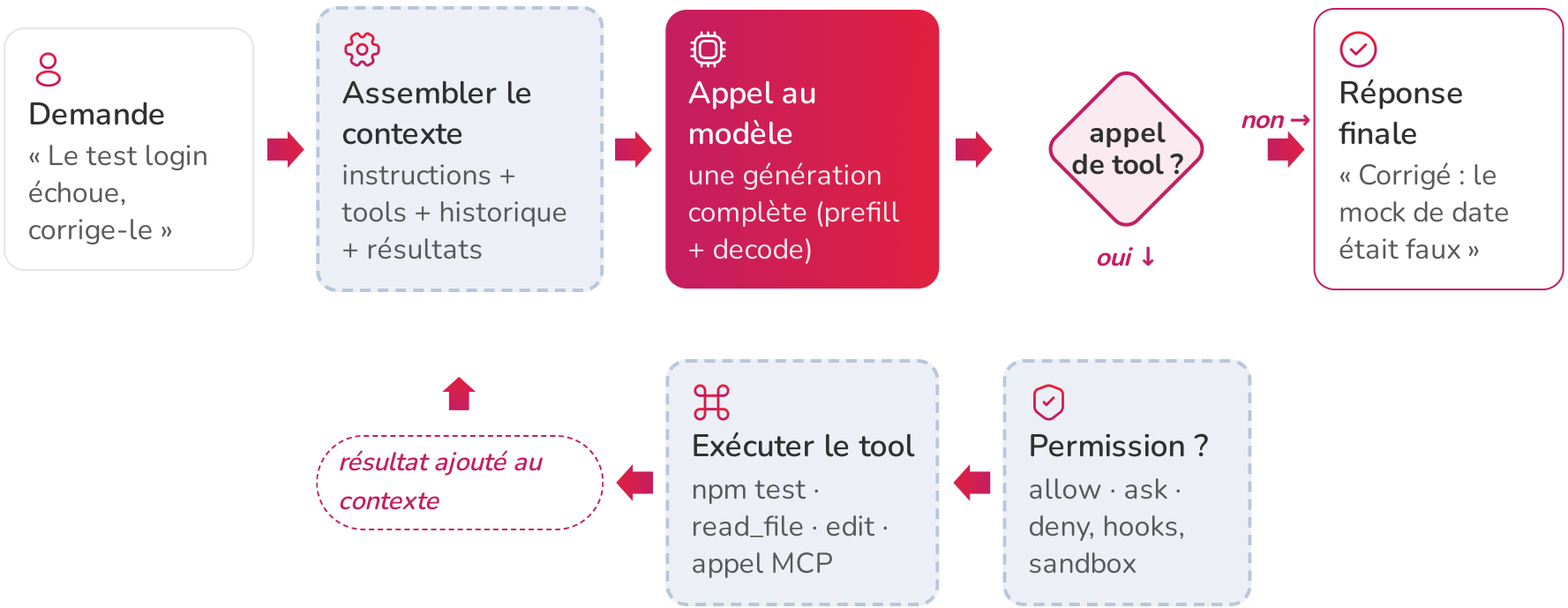
Objectif

Et un critère d'arrêt.

Les noms de cette boucle : **ReAct** (*Reason + Act*, 2022), *gather context* → *take action* → *verify work* → *repeat* (Anthropic), OODA. Même mécanisme.

© ZENIKA 2026 | All rights reserved

La boucle, tour par tour



Un tour = un appel modèle complet. Une tâche simple : 5 à 20 tours. Une grosse tâche : des centaines.

© ZENIKA 2026 | All rights reserved

Tool calling : ce qui transite réellement

① Le harnais déclare l'outil

```
{
  "name": "run_tests",
  "description":
    "Lance les tests",
  "input_schema": {
    "type": "object",
    "properties": {
      "path": {
        "type": "string"
      }
    }
  }
}
```

② Le modèle demande

```
{
  "type": "tool_use",
  "id": "call_01",
  "name": "run_tests",
  "input": {
    "path": "tests/login"
  }
}
```

③ Le harnais répond

```
{
  "type": "tool_result",
  "tool_use_id": "call_01",
  "content":
    "FAIL login:42
     expected 200,
     got 401"
}
```

i Le modèle choisit **sur la description** : nom, description et schéma de chaque outil sont envoyés **en tokens** à chaque tour. 40 outils MCP branchés = des milliers de tokens de préfixe.

© ZENIKA 2026 | All rights reserved

Mémoire : quatre étages

1



Fenêtre

La conversation en cours. Rapide, chère, limitée.

RÔLE
mémoire de travail

2



Compaction

Quand la fenêtre sature, le harnais résume l'historique et repart du résumé.

RÔLE
résumé

3



Fichiers

AGENTS.md, CLAUDE.md, notes, plans : relus à chaque session.

RÔLE
mémoire persistante

4



Recherche

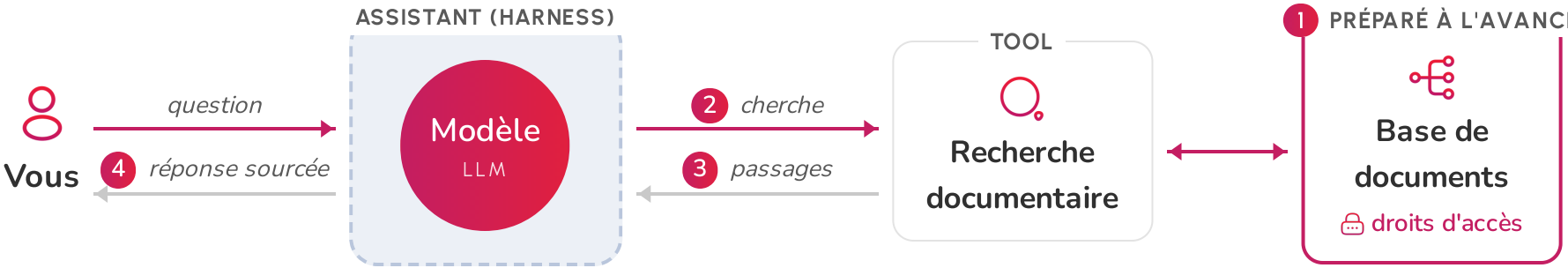
grep, index sémantique (RAG), MCP : on va chercher seulement ce qui sert.

RÔLE
mémoire à la demande

i Les agents de code privilégient souvent la **recherche agentique** (`grep` , `find` , lecture de fichiers) plutôt qu'un index vectoriel : toujours à jour, et le code se cherche bien par mots exacts.

© ZENIKA 2026 | All rights reserved

RAG : le retour des embeddings



Exemple : « Comment configure-t-on le retry du client HTTP interne ? »

1

Indexer *(indexing)*

La doc est découpée en morceaux ; un modèle d'embedding calcule un vecteur par morceau.

> *Wiki technique et ADR indexés dans pgvector.*

2

Chercher *(retrieval)*

La question est vectorisée ; on prend les morceaux les plus proches.

> *Les 3 passages sur la politique de retry.*

3

Augmenter *(augmentation)*

Ces morceaux entrent dans le contexte, avec la question.

> *3 passages, pas les 400 pages du wiki.*

4

Générer *(generation)*

Le modèle répond à partir des passages et cite ses sources.

> *« maxRetries = 3, backoff exponentiel [ADR-012] »*

© ZENIKA 2026 | All rights reserved

MCP : une prise standard pour les outils

MODEL CONTEXT PROTOCOL

L'« USB-C » des agents

Protocole ouvert lancé par Anthropic fin 2024, adopté par OpenAI, Google, Microsoft, GitHub. Un serveur MCP écrit une fois fonctionne dans tous les harnais compatibles.

> *Transport : stdio (local) ou HTTP (distant), JSON-RPC.*

CE QU'EXPOSE UN SERVEUR

Tools (actions), Resources (données à lire), Prompts (modèles réutilisables).

EXEMPLES DE SERVEURS

GitHub · Playwright · PostgreSQL · Atlassian · Figma · Sentry

LE REVERS

Chaque serveur ajoute des définitions d'outils au contexte, et c'est du code tiers qui agit avec vos droits. Un serveur malveillant ou une donnée piégée dans sa réponse est un vecteur d'attaque (partie 5).

ET ENTRE AGENTS ?

A2A (Agent2Agent, initié par Google) : un protocole pour que des agents de fournisseurs différents se découvrent et se délèguent des tâches. MCP connecte un agent à des outils ; A2A connecte des agents entre eux.

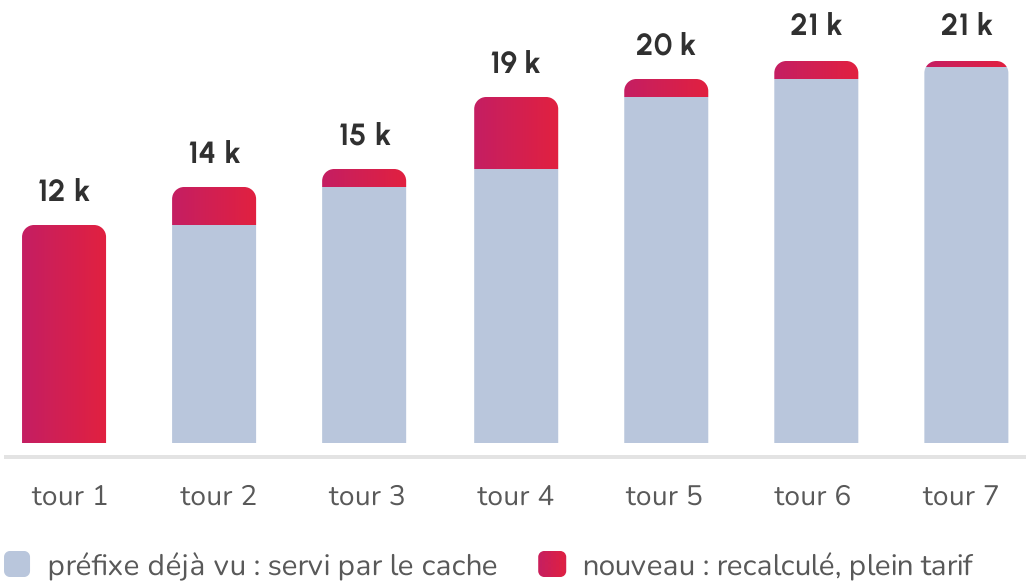
© ZENIKA 2026 | All rights reserved

Une trace réelle, commentée

Tour	Le modèle demande	Le harnais renvoie	Contexte
1	`bash: npm test -- login`	échec : `expected 200, got 401`	12 k
2	`read_file: tests/login.spec.ts`	80 lignes	14 k
3	`grep: "createSession"`	3 occurrences	15 k
4	`read_file: src/auth/session.ts`	210 lignes	19 k
5	`edit: session.ts` (2 lignes)	diff appliqué, après accord	20 k
6	`bash: npm test -- login`	succès : `1 passed`	21 k
7	*pas d'outil* : réponse finale	« Corrigé : le token expirait »	21 k

© ZENIKA 2026 | All rights reserved

Ce que coûte cette trace



122 k tokens d'entrée envoyés au total, dont 83 % lus en cache

LE CONSTAT

Le préfixe déjà vu domine

Chaque tour ne rajoute que quelques milliers de tokens : tout le reste a déjà été lu au tour précédent.

AVEC LE PROMPT CACHING

Ce préfixe est lu en cache, facturé ~10 % : sans cache, ce bug simple coûterait plusieurs fois plus cher.

© ZENIKA 2026 | All rights reserved

Comment un agent déraille

BOUCLE SANS FIN

Même correction tentée dix fois, sans progrès. *Parade* : limite de tours, détection de répétition.

DÉRIVE DE CONTEXTE

Trop d'informations, l'objectif initial se dilue. *Parade* : sessions courtes, sous-agents, plan écrit.

FAUX SUCCÈS

« Tous les tests passent »… après avoir supprimé le test. *Parade* : vérification indépendante, revue de diff.

OUTIL HALLUCINÉ

Appel d'une fonction ou d'une option CLI qui n'existe pas. *Parade* : schémas stricts, messages d'erreur utiles.

EXPLOSION DE COÛT

Un agent autonome laissé la nuit. *Parade* : budget par tâche, alertes, gateway.

INJECTION INDIRECTE

Un fichier, une page web ou une issue contient des instructions que l'agent suit. **Partie 5.**

© ZENIKA 2026 | All rights reserved

Partie 4 · 12 min

Harnais & orchestrateurs

Le produit qui implémente la boucle, puis ce qu'on ajoute quand un seul agent ne suffit plus.



© ZENIKA 2026 | All rights reserved

Anatomie d’un harnais

DÉFINITION

Tout le code qui n'est pas le modèle

Même modèle, deux harnais : des résultats et des coûts **très différents**. Le harnais est un levier de performance au même titre que le choix du modèle.

> *« Agent de code = modèle + harnais »*

BOUCLE

Tours, arrêt, reprise, gestion d'erreurs.

OUTILS INTÉGRÉS

Fichiers, shell, recherche, web, MCP.

CONTEXTE

Assemblage, compaction, cache, mémoire.

PERMISSIONS

allow / ask / deny, sandbox, conteneur.

HOOKS

Scripts déterministes avant / après chaque action.

SOUS-AGENTS

Déléguer avec un contexte isolé.

OBSERVABILITÉ

Traces, coût par session, rejouer les décisions.

EXTENSIONS

Skills, commandes, plugins, instructions projet (`AGENTS.md`).

© ZENIKA 2026 | All rights reserved

Le marché des harnais

TERMINAL + IDE, ÉDITEUR DU MODÈLE

Intégrés verticalement

Claude Code (Anthropic)

Codex (OpenAI) Gemini CLI (Google)

Mistral Vibe

PLATEFORME DEV, MULTI-MODÈLES

Intégrés à l'outillage

GitHub Copilot (IDE, CLI, cloud)

Cursor Kiro (AWS)

JetBrains Junie

OPEN SOURCE, MODÈLE AU CHOIX

Agnostiques

OpenCode Aider Cline

OpenHands Goose

POUR CONSTRUIRE VOTRE PROPRE HARNAIS

SDK d'agents

La boucle, les outils et le contexte en bibliothèque : on écrit l'agent métier, pas la plomberie.

Claude Agent SDK OpenAI Agents SDK Google ADK Vercel AI SDK Pydantic AI

AGENTS CLOUD

Le harnais tourne ailleurs

Copilot coding agent, Codex cloud, Devin, Jules : on assigne une issue, on reçoit une PR.

© ZENIKA 2026 | All rights reserved

Cinq patterns d'orchestration

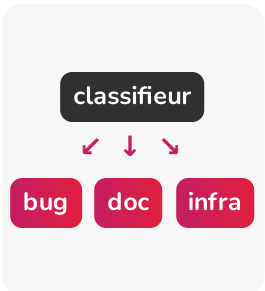
Chaîne



étapes fixes, chaque sortie nourrit la suivante

> *spec → plan → code*

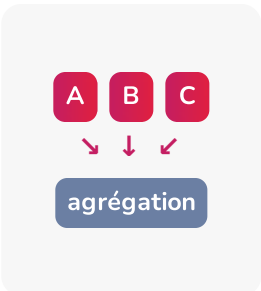
Routeur



on oriente vers le bon agent ou le bon modèle

> *un SLM trie, un LLM traite*

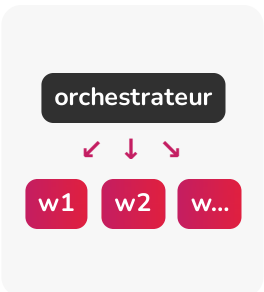
Parallélisation



tâches indépendantes, ou vote entre plusieurs

> *3 revues d'une même PR*

Orchestrateur-workers



le découpage est décidé dynamiquement par un LLM

> *refacto sur 12 modules*

Évaluateur



on itère jusqu'à ce que le critère soit atteint

> *code + revue par un 2e agent*

workflow figé : chemin connu à l'avance

workflow dynamique : le LLM pilote davantage

© ZENIKA 2026 | All rights reserved

Le marché des orchestrateurs

FRAMEWORKS D'AGENTS (CODE)

On décrit le graphe

- LangGraph : graphe d'états, reprise, humain dans la boucle
- CrewAI : équipes de rôles
- Microsoft Agent Framework : successeur d'AutoGen et Semantic Kernel
- Google ADK, OpenAI Agents SDK (handoffs)
- LlamaIndex, Mastra (TypeScript)

EXÉCUTION DURABLE

Ne jamais perdre une tâche

Reprise après crash, tâches de plusieurs heures, retries.

Temporal · Inngest · Restate

LOW-CODE

Glisser-déposer

n8n · Dify · Flowise · Langflow

PLATEFORMES CLOUD MANAGÉES

Agents hébergés

AWS Bedrock AgentCore

Vertex AI Agent Engine

Azure AI Foundry Agents

DANS LE HARNAIS

Orchestration intégrée

Sous-agents, agents parallèles en worktrees, workflows : Claude Code, Copilot, Codex.

© ZENIKA 2026 | All rights reserved

Et le SDD dans tout ça ?

1



Spécifier

Le quoi et le pourquoi, relu par un humain.

ARTEFACT

spec.md

2



Planifier

Le comment technique.

ARTEFACT

plan.md

3



Découper

De petites tâches vérifiables.

ARTEFACT

tasks.md

4



Implémenter

Un agent (ou des workers) par tâche.

ARTEFACT

code + tests

i Le SDD n'orchestre pas des processus mais des **artefacts** relus par un humain : une **chaîne** dont chaque sortie est un fichier versionné. Outillage : **GitHub Spec Kit**, **Kiro**, **OpenSpec**, **BMAD Method**.

© ZENIKA 2026 | All rights reserved

Partie 5 · 15 min

Guardrails & gateways

Des agents qui lisent, exécutent et appellent des API : comment l'entreprise garde le contrôle ?



© ZENIKA 2026 | All rights reserved

Ce dont on se protège

MENACE N° 1 · OWASP LLM01

Injection de prompt

Directe : l'utilisateur tente de détourner l'agent. **Indirecte** : l'instruction malveillante est cachée dans une donnée lue par l'agent (README, issue GitHub, page web, réponse MCP).

> *Pour le modèle, tout est du texte : il ne distingue pas fiablement une donnée d'une consigne.*

FUITE DE DONNÉES

Secrets, PII, code propriétaire envoyés au modèle ou exfiltrés.

AGENTIVITÉ EXCESSIVE

`rm -rf`, `git push --force`, prod modifiée.

SUPPLY CHAIN

Serveur MCP, skill ou modèle compromis.

CONSOMMATION NON BORNÉE

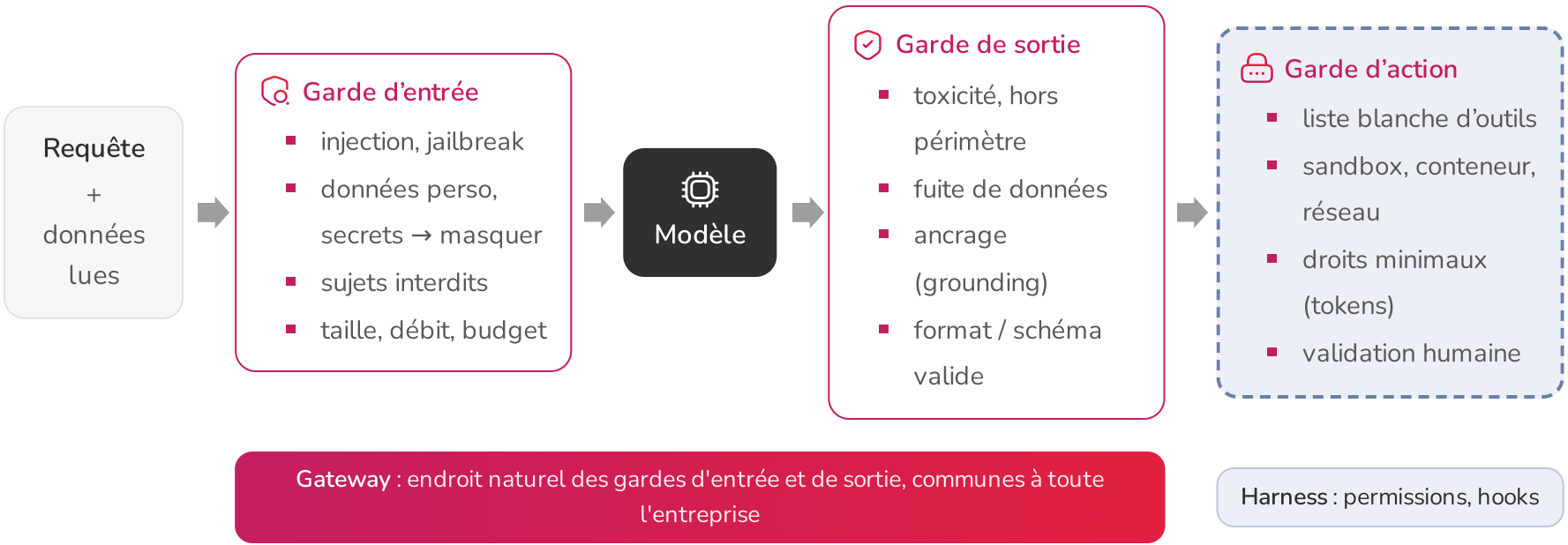
Boucles, abus, facture qui explose.

LE « TRIO LÉTAL » (SIMON WILLISON)

Un agent qui cumule ① l'accès à des données privées, ② l'exposition à du contenu non fiable et ③ un moyen de communiquer vers l'extérieur peut être amené à exfiltrer vos données. Règle pratique : casser au moins une des trois branches.

© ZENIKA 2026 | All rights reserved

Où placer les contrôles



Les gardes d'entrée et de sortie **réduisent** le risque (des classifieurs, faillibles). La garde d'action le **borne** : un agent sans accès réseau ne peut pas exfiltrer.

© ZENIKA 2026 | All rights reserved

Les fournisseurs de guardrails

Offre	Nature	Ce qu'elle apporte	Où ça tourne
AWS Bedrock Guardrails	service managé	filtres, sujets refusés, masquage PII, ancrage	AWS (API aussi hors Bedrock)
Azure AI Content Safety	service managé	*Prompt Shields* (injections directes et indirectes), ancrage	Azure
Google Model Armor	service managé	injection, jailbreak, données sensibles, URL malveillantes	Google Cloud
NVIDIA NeMo Guardrails	open source	« rails » programmables (Colang) : dialogue, sujets, outils	auto-hébergé
Guardrails AI	open source	validateurs : PII, format, toxicité…	dans votre code
Llama Guard / Prompt Guard	modèles ouverts	classifieurs de sécurité et d'injection	auto-hébergé (SLM)
OpenAI · Mistral Moderation	API	classification de contenus à risque	cloud du fournisseur
Lakera Guard	SaaS spécialisé	détection d'injection en temps réel	SaaS ou auto-hébergé

© ZENIKA 2026 | All rights reserved

La gateway IA : un seul point de passage

DÉFINITION

Un reverse proxy spécialisé entre vos applications et les fournisseurs de modèles

Une seule API (souvent compatible OpenAI) côté applications, N fournisseurs derrière. Les clés des fournisseurs ne quittent jamais la gateway.

ROUTAGE

Choix du modèle, répartition de charge, fallback si un fournisseur tombe.

FINOPS

Budgets et quotas par équipe, par clé, par projet.

CACHE

Exact ou sémantique : même question, réponse sans appel.

OBSERVABILITÉ

Logs, traces, coût et latence par appel.

GUARDRAILS BRANCHÉS

Masquage PII, détection d'injection, filtres de sortie, appliqués à tous sans toucher aux applications.

CONFORMITÉ

Résidence des données, liste de modèles autorisés, piste d'audit (AI Act, RGPD).

© ZENIKA 2026 | All rights reserved

Les fournisseurs de gateways

Offre	Nature	Point fort	Pour qui
LiteLLM	open source	API OpenAI vers 100+ fournisseurs, budgets	auto-héberger vite
Portkey	SaaS + open source	routage, cache, guardrails, observabilité	produits IA en prod
Kong AI Gateway	plugins Kong	limites en tokens, cache, garde de prompt	déjà sur Kong
Azure APIM	service managé	quotas de tokens, cache sémantique	entreprises sur Azure
Cloudflare AI Gateway	SaaS, en périphérie	une URL : logs, cache, limites de débit	démarrer sans infra
Envoy AI Gateway	open source	basé sur Envoy, natif Kubernetes	plateformes cloud native
OpenRouter	agrégateur SaaS	une API, une facture, des centaines de modèles	explorer, comparer

© ZENIKA 2026 | All rights reserved

Brancher un agent de code sur la gateway

Claude Code

```
# parle à la gateway, pas à Anthropic
export ANTHROPIC_BASE_URL=\
  "https://llm-gw.example"
export ANTHROPIC_AUTH_TOKEN=\
  "$CLE_EQUIPE_FRONT"
claude
```

Ce que l'entreprise gagne : une clé par équipe, un budget, des logs, le masquage des secrets, la liste des modèles autorisés.

Codex · `~/.codex/config.toml`

```
model_provider = "gw"
[model_providers.gw]
name     = "Gateway interne"
base_url = "https://llm-gw.example/v1"
env_key  = "CLE_EQUIPE_FRONT"
```

Ce qui reste côté poste : la gateway ne voit que les appels au modèle, pas les commandes exécutées.

> *i* Le développeur ne voit jamais la clé du fournisseur. Permissions du harnais, sandbox et hooks restent **indispensables**.

© ZENIKA 2026 | All rights reserved

À vous : on en discute

1



Vos agents cumulent-ils le trio létal ?

Données privées, contenu non fiable, sortie réseau.

PISTE

casser au moins une branche

2



Qui voit vos prompts aujourd'hui ?

Fournisseur, agrégateur, gateway, logs : listez les maillons.

PISTE

cartographier la chaîne

3



Gateway managée ou auto-hébergée ?

Et quel premier guardrail activeriez-vous ?

PISTE

commencer par les secrets et PII

© ZENIKA 2026 | All rights reserved

Une requête, de bout en bout

1



SDD
Une tâche issue d'une spec relue.

2



Orchestration
La confie à un agent.

3



Harnais
Contexte, préfixe stable en tête.

4



Gateway
Filtre, route, journalise.

5



Modèle
Tokenise, prefill en cache, decode.

6



Harnais
Permission, outil, reboucle.

LES TROIS IDÉES À GARDER

Le modèle prédit, le harnais agit, la gateway gouverne

La qualité dépend du **contexte** fourni. Le coût dépend des **tokens** et du **cache**. La sécurité dépend de ce que le harnais **autorise** à exécuter.



POUR ALLER PLUS LOIN

- *Building effective agents*, Anthropic (2024)
- *Attention Is All You Need*, Vaswani et al. (2017)
- « Intro to LLMs », Andrej Karpathy · OWASP Top 10 for LLM (2025)

© ZENIKA 2026 | All rights reserved

Des questions ?

Le modèle prédit, le harnais agit, la gateway gouverne.

© ZENIKA 2026 | All rights reserved